Branch/Semester	B.Tech CSE (AI & ML) / Semester V
Subject Name:	Design and Analysis of Algorithms
Subject Code:	01AI0506
Assignment:	SMF Assignment
Date:	19th October 2026

Important Instructions:

- Students must solve the assignment according to their Mid-1 marks category.
- **Below 12 marks**: Easy-level assignment (Slow Learners).
- **12–18 marks**: Medium-level assignment (Medium Learners).
- **Above 18 marks**: Advanced-level assignment (Fast Learners).
- All 10 questions must be solved in the student's own handwriting.
- Write complete C++/Java/Python programs wherever required.
- Show all steps for numerical problems and time complexity analysis.
- Submit the assignment as a handwritten hard copy within the deadline specified by the subject faculty.

Sr.	Slow Learners	CO	BL
1	Apply the concept of an algorithm to write step-by-step instructions for finding the larger of two numbers.	CO1	BL3
2	Analyze the number of comparisons performed by linear search when the required element is at the first position and at the last position.	CO2	BL4
3	Apply bubble sort to arrange the numbers 5, 2, 4, 1 in ascending order. Show each pass.	CO2	BL3
4	Compare linear search and binary search for finding an element in a sorted list. Select the more efficient method and justify your choice.	CO2	BL5
5	Apply the dynamic programming approach to the Matrix Chain Multiplication problem for three matrices A1 (10 × 30), A2 (30 × 5), and A3 (5 × 60). Determine the optimal parenthesization and the minimum number of scalar multiplications required.	CO3	BL3
6	Apply the greedy strategy to select the maximum number of non-overlapping activities from the following: A(1,2), B(2,4), C(3,5), D(5,7). Assume activities are represented by (start time, finish time).	CO4	BL3

7	Analyze BFS and DFS by identifying the main difference in how each visits vertices of a graph.	CO1	BL4
8	Apply the backtracking approach to the 4-Queens problem and identify one valid arrangement of four queens on a chessboard.	CO1	BL3
9	Classify the following tasks as tractable or potentially intractable based on their known complexity: sorting n numbers using merge sort and solving the general travelling salesperson problem exactly. Give reasons.	CO3	BL4
10	Evaluate why an approximation algorithm may be preferred over an exact algorithm for a large travelling salesperson problem.	CO4	BL5

Sr.	**Medium Learners**	**CO**	**BL**
1	Analyze the best-case, average-case, and worst-case time complexity of linear search. Illustrate each case using a suitable example.	CO2	BL4

2	Analyze the time complexity of the following code and express it using asymptotic notation:
	for (int i = 1; i <= n; i++) {
	cout << i << endl;
	}
	Justify your answer.
		CO2	BL4

3	Apply insertion sort to arrange 25, 12, 18, 8, 20 in ascending order. Show each pass and the final sorted sequence.	CO2	BL3
4	Evaluate merge sort and quicksort in terms of their best-case, average-case, and worst-case time complexities. Recommend a suitable algorithm for a large dataset and justify your choice.	CO2	BL5
5	Apply dynamic programming to the 0/1 knapsack problem with capacity 5, item weights (2, 3, 4), and corresponding values (3, 4, 5). Construct the DP table and find the maximum value.	CO4	BL3
6	Apply Kruskal's algorithm to find the minimum spanning tree of a graph with edges AB = 1, AC = 3, BC = 2, BD = 4, and CD = 5. Show the selected edges and total weight.	CO4	BL3
7	Analyze Dijkstra's algorithm for a graph with edges A–B = 4, A–C = 1, C–B = 2, B–D = 1, and C–D = 5. Find the shortest distances from A to all other vertices.	CO2	BL4

8	Apply backtracking to the subset sum problem for the set 5 with target sum 9. Identify all valid subsets and show the search decisions.	CO2	BL3
9	Analyze the relationship among P, NP, NP-hard, and NP-complete problems. Explain how polynomial-time reductions help establish NP-completeness.	CO3	BL4
10	Evaluate the suitability of an approximation algorithm for the bin packing problem when the number of items is very large. Discuss solution quality and computational efficiency.	CO5	BL5

Sr.	**Fast Learners**	**CO**	**BL**
1	Analyze the recurrence T(n)=2T(n/2)+n, with T(1)=1, using the recurrence tree or Master Theorem. Derive the asymptotic complexity and justify the result.	CO2	BL4

2	Evaluate the efficiency of the following code by deriving its time complexity:
	`for (int i = 1; i <= n; i++) {`
	`  int j = 1;`
	`  while (j < n) {`
	`  j = 2 * j;`
	`  }`
	`}`
	Justify the final asymptotic bound.
		CO2	BL5

3	Apply quicksort to the array 35, 12, 48, 7, 25, 19, 40 using the first element as pivot. Show the partitioning steps and analyze its best-case and worst-case complexity.	CO2	BL4
4	Construct a max heap for the array 16, 4, 10, 14, 7, 9, 3, 2, 8, 1. Apply heap sort and evaluate its time and space complexity.	CO2	BL5
5	Apply dynamic programming to find a longest common subsequence (LCS) of the strings ABCBDAB and BDCABA. Construct the DP table and determine the LCS length and one valid subsequence.	CO4	BL3
6	Apply the fractional knapsack algorithm to a knapsack of capacity 50 with items (weight, value): (10,60), (20,100), and (30,120). Determine the	CO4	BL5

	optimal selection and maximum value, and justify why the greedy strategy works for this problem.		
7	Apply the Floyd–Warshall algorithm to the following initial distance matrix and derive the final all-pairs shortest-distance matrix: **0** **4** ∞ ∞ **0** **2** **3** ∞ **0** Show the intermediate updates.	CO2	BL4
8	Apply backtracking to solve the 4-Queens problem. Show a valid arrangement and analyze how backtracking eliminates invalid states.	CO2	BL4
9	Evaluate the role of polynomial-time reductions in proving NP-completeness. Explain the conditions needed for a valid reduction and illustrate them using a suitable example.	CO3	BL5
10	Evaluate randomized quicksort and randomized primality testing in terms of expected performance, correctness, and practical applications. Compare randomized and deterministic algorithms.	CO5	BL5